AI_HW3

1、简答题

1.1

交叉熵（Cross Entropy）是一种常用的损失函数，特别适用于分类任务。它衡量的是两个概率分布之间的距离，常用于比较预测的概率分布 $\hat{y}$ 与真实的类别分布 y （通常是one-hot编码）的差异。

交叉熵直接衡量两个概率分布之间的“距离”，绝对值损失只是在数值上做逐维差异的加总，没有考虑概率的语义结构，不适合用于分类任务。交叉熵对正确类别的置信度更敏感，绝对值损失只看每个元素之间的绝对差值，不会特别强调正确类别的置信度。交叉熵能推动概率分布朝向 one-hot。交叉熵结合 softmax 输出后，其梯度表达式简洁、稳定，这使得反向传播更高效，训练更稳定。在实际深度学习任务中（如图像分类、文本分类），使用交叉熵损失的模型通常：收敛更快；更容易达到高准确率；更稳定，而 L1 损失容易使训练停滞在非最优解上，尤其在高维概率输出空间中。

1.2

非线性建模能力更强；能表示特征之间的交互，线性模型默认特征之间是独立线性作用，而 MLP 可通过多层非线性组合隐式地建模特征交互；图像分类、语音识别、自然语言处理等任务，往往存在高度复杂的模式，线性模型几乎无能为力，而 MLP 可以提取抽象表示、捕捉深层特征。

1.3

卷积：

$$(I * K)(i,j) = \sum_m \sum_n I(i-m, j-n) \cdot K(m,n)$$

互相关：

$$(I * K)(i,j) = \sum_m \sum_n I(m,n) \cdot K(i-m, j-n)$$

CNN 中的“卷积”操作用的几乎都是互相关。

1.4

如果优化器是朴素的随机梯度下降（SGD，无动量、无自适应学习率），并且模型中没有BatchNorm等依赖于batch统计量的层，那么梯度累积两次再进行反向传播的方法可以确保训练得到的模型效果和参数与直接使用完整batch训练一致。原因是朴素的SGD中梯度是线性可加的，且梯度累积后的参数更新与直接使用完整batch的更新完全一致。

如果优化器使用了动量（如SGD with Momentum）或自适应学习率（如Adam），则梯度累积会导致训练不一致。 因为动量或自适应学习率的更新会在每个子batch后调整，而直接使用完整batch时只调

整一次，导致梯度更新行为不同。

1.5

残差连接通过引入恒等映射（identity mapping）构建快捷路径（shortcut path），有效缓解梯度消失问题，提升梯度流动性，简化目标函数的优化，从而显著改善深层神经网络的可训练性与收敛稳定性。

残差链接能够缓解梯度消失（GradientVanishing）的问题。

2、解答题

2.1 卷积神经网络

2.1.1

$$5 \times 5 \times 1 \times 3 = 75$$

2.1.2

$$\lfloor \frac{55-5}{2} \rfloor + 1 = 26$$
$$\lfloor \frac{26-2}{2} \rfloor + 1 = 13$$
$$\Longrightarrow 13 \times 13 \times 3$$

2.1.3

$$13 \times 13 \times 3 = 507$$

2.1.4

$$ReLU: f(x) = max(0, x)$$
$$Sigmoid: f(x) = \frac{1}{1+e^{-x}}$$

2.2 注意力机制

2.2.1

$$y_i = \sum_{j=1}^{i} \alpha_{ij} v_j, \alpha_{ij} = \frac{exp(\frac{q_i^T k_j}{\sqrt{d}})}{\sum_{t=1}^{i} exp(\frac{q_i^T k_t}{\sqrt{d}})}$$

2.2.2

$$q_3 = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ 1 \\ -1 \\ 0 \end{bmatrix}, y_3 = \frac{exp(-\frac{\sqrt{2}}{4})v_1 + exp(-\frac{\sqrt{2}}{4})v_2 + exp(\frac{\sqrt{2}}{2})v_3}{exp(\frac{\sqrt{2}}{2}) + 2exp(-\frac{\sqrt{2}}{4})}$$

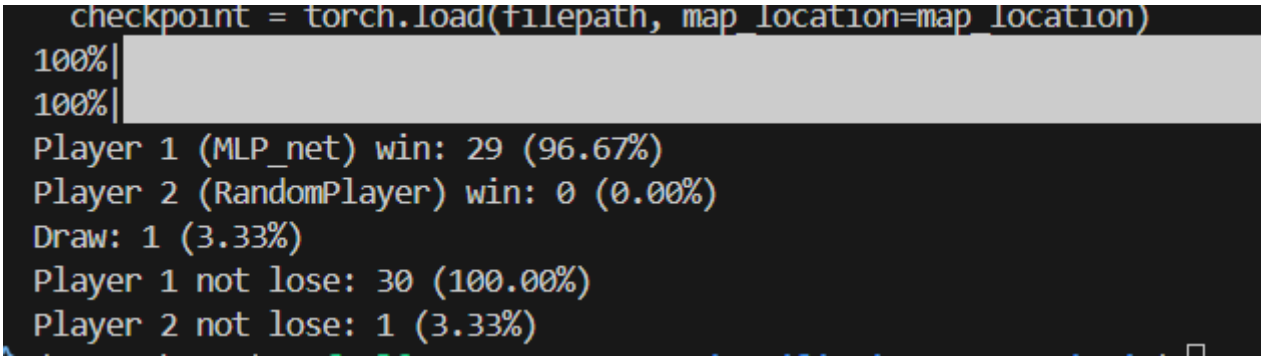
2.2.3

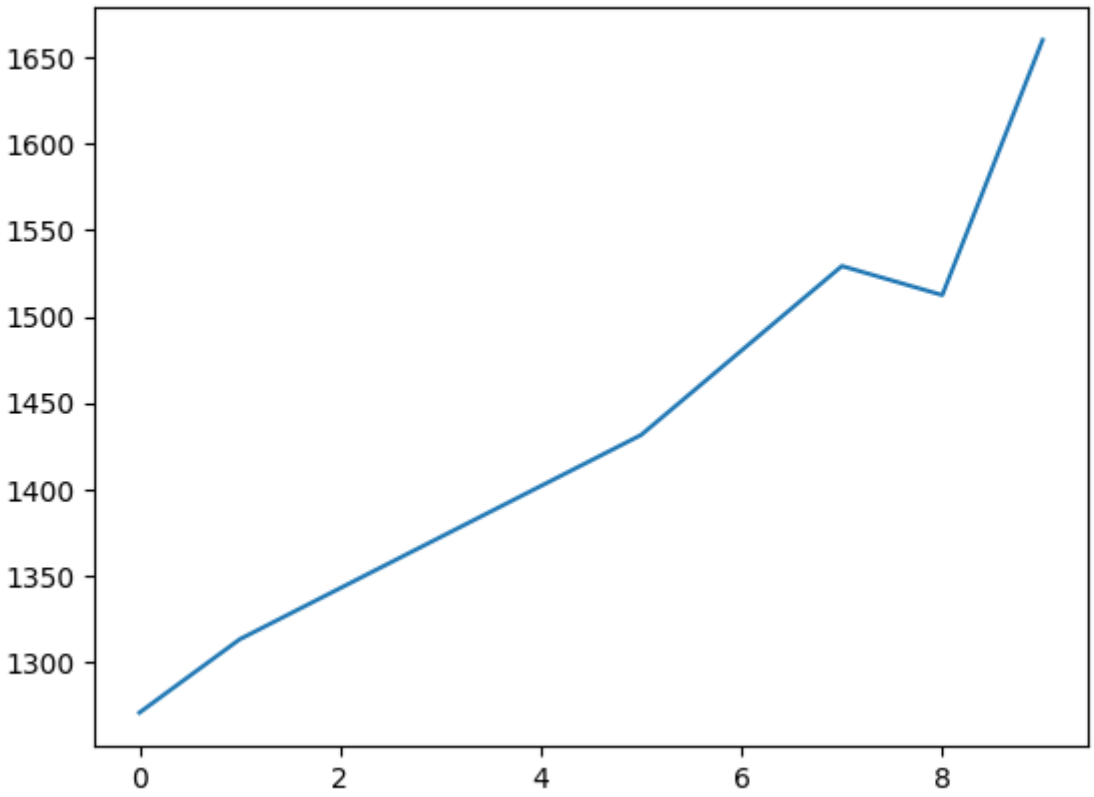
$$q_4 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 0 \\ -1 \\ -1 \end{bmatrix}, y_4 = \left[y_4^{(1)}; y_4^{(2)}\right] = (\sum_{j=1}^{4} \alpha_{4j}^{(1)} v_j^{(1)}) \oplus (\sum_{j=1}^{4} \alpha_{4j}^{(2)} v_j^{(2)}) =$$

$$\frac{exp(\frac{\sqrt{3}}{3})v_1 + exp(-\frac{\sqrt{3}}{3})v_2 + exp(\frac{\sqrt{3}}{3})v_3 + exp(\frac{\sqrt{3}}{3})v_4}{3exp(\frac{\sqrt{3}}{3}) + exp(-\frac{\sqrt{3}}{3})}$$

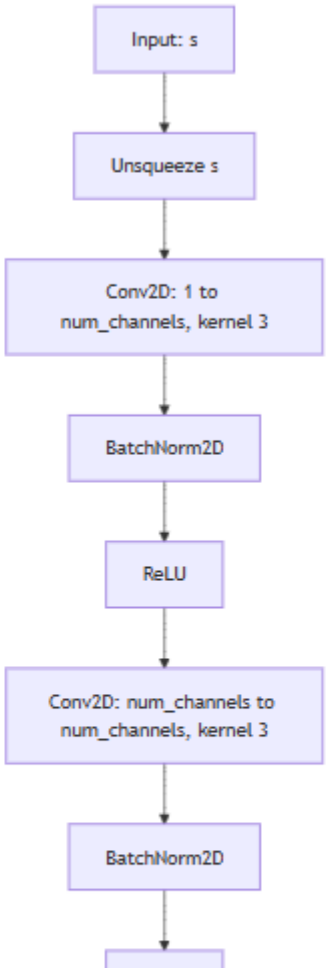
3、深度学习与 AlphaZero

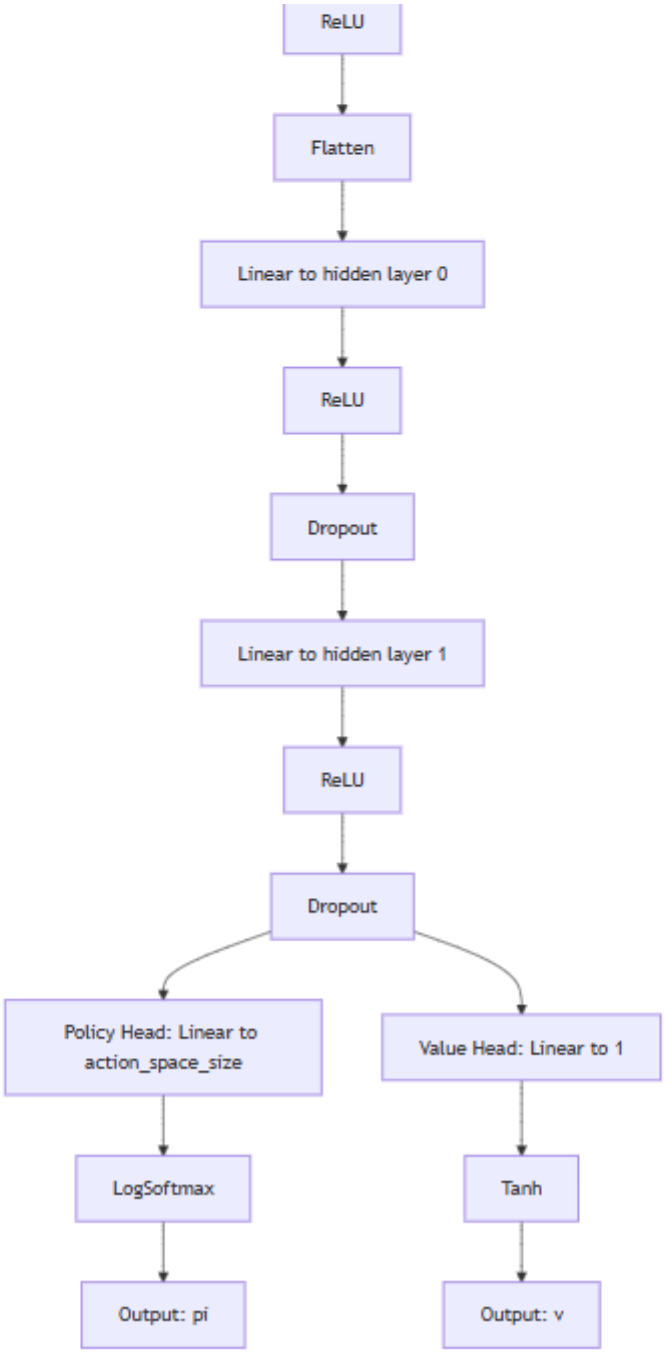
3.1




1650
1600
1550
1500
1450
1400
1350
1300
0
2
4
6
8

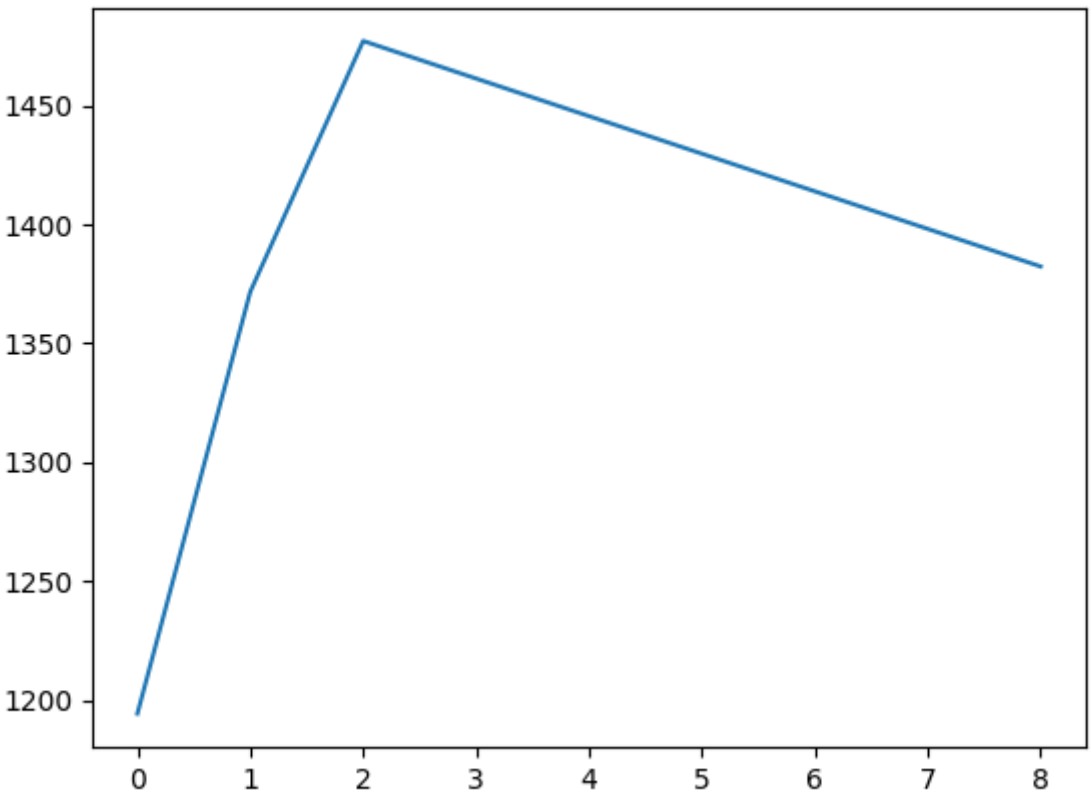
3.2


Input: s
Unsqueeze s
Conv2D: 1 to num_channels, kernel 3
BatchNorm2D
ReLU
Conv2D: num_channels to num_channels, kernel 3
BatchNorm2D


ReLU
Flatten
Linear to hidden layer 0
ReLU
Dropout
Linear to hidden layer 1
ReLU
Dropout
Policy Head: Linear to action_space_size
Value Head: Linear to 1
LogSoftmax
Tanh
Output: pi
Output: v

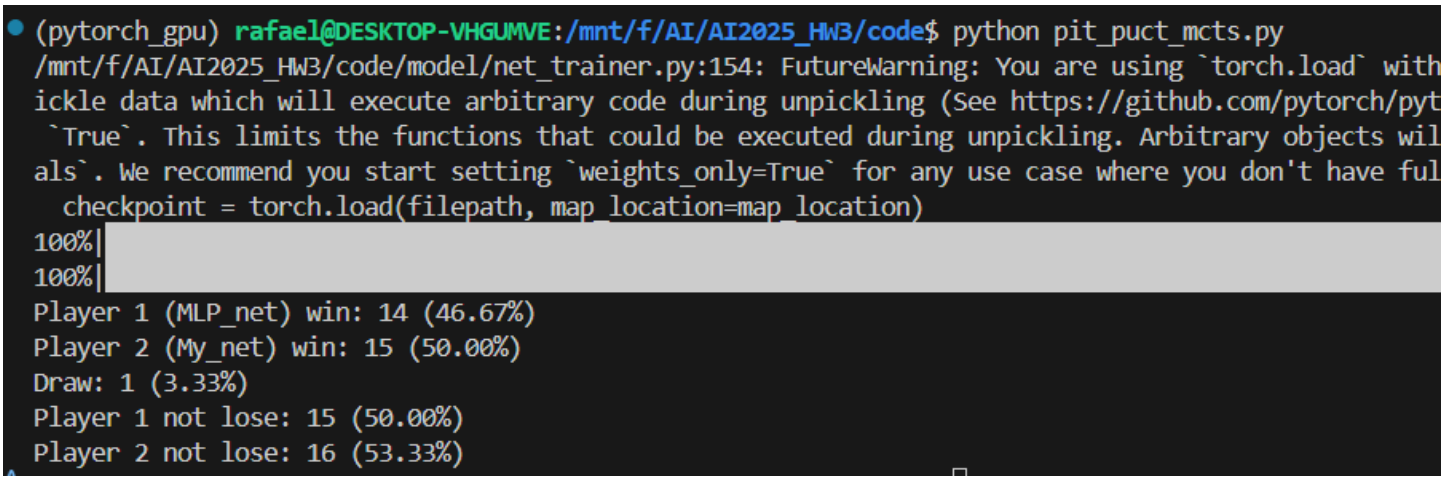
3.3

```
------ Start Self-Play Iteration 9 ------
[AlphaZeroParallel] Finished 20 episodes (13280 examples) in 134.391s, Win: 9, Draw: 0, Lose: 11
[TRAIN DATA SIZE]: 63370
Training Net: 100%|
[AlphaZeroParallel] Start evaluating with last best model for 20 round
[EVALUATION RESULT]: currrent_win12, last_win6, draw2; win_rate=0.667
[ACCEPT NEW MODEL]
[AlphaZeroParallel] Start evaluating with baseline for 20 round
[EVALUATION RESULT]: win19, lose0, draw1
[EVALUATION RESULT]:(first)  win9, lose0, draw1
[EVALUATION RESULT]:(second) win10, lose0, draw0
------ Finished Self-Play Iteration 9 in 345.344s ------
```




```
(pytorch_gpu) rafael@DESKTOP-VHGUMVE:/mnt/f/AI/AI2025_HW3/code$ python pit_puct_mcts.py
/mnt/f/AI/AI2025_HW3/code/model/net_trainer.py:154: FutureWarning: You are using `torch.l
ickle data which will execute arbitrary code during unpickling (See https://github.com/py
 `True`. This limits the functions that could be executed during unpickling. Arbitrary ob
als`. We recommend you start setting `weights_only=True` for any use case where you don't
  checkpoint = torch.load(filepath, map_location=map_location)
100%|
100%|
Player 1 (My_net) win: 28 (93.33%)
Player 2 (RandomPlayer) win: 2 (6.67%)
Draw: 0 (0.00%)
Player 1 not lose: 28 (93.33%)
Player 2 not lose: 2 (6.67%)
```

3.4


(pytorch_gpu) rafael@DESKTOP-VHGUMVE:/mnt/f/AI/AI2025_HW3/code$ python pit_puct_mcts.py
/mnt/f/AI/AI2025_HW3/code/model/net_trainer.py:154: FutureWarning: You are using `torch.load` with
ickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pyt
`True`. This limits the functions that could be executed during unpickling. Arbitrary objects wil
als`. We recommend you start setting `weights_only=True` for any use case where you don't have ful
checkpoint = torch.load(filepath, map_location=map_location)
100%|
100%|
Player 1 (MLP_net) win: 14 (46.67%)
Player 2 (My_net) win: 15 (50.00%)
Draw: 1 (3.33%)
Player 1 not lose: 15 (50.00%)
Player 2 not lose: 16 (53.33%)